

ภาคผนวก

ภาคผนวก ก คู่มือการติดตั้ง

ภาคผนวก ข คู่มือการใช้งาน

เริ่มต้นด้วย THL

ให้คุณลองสร้างไฟล์ *hello.thl* แล้วเขียนโค้ดนี้ลงไป

```
แสดง("helloTHL")
```

จากนั้นเปิด command line แล้วไปยังไดเรกทอรีที่เก็บไฟล์อยู่ จากนั้นลองพิมพ์คำสั่ง:

```
mmt hello.thl
```

ผลลัพธ์ควรจะได้:

```
helloTHL
```

จากตัวอย่างข้างต้นสรุปได้ว่าหากจะรันโปรแกรมให้ใช้คำสั่ง *mmt* ตามด้วยชื่อไฟล์ที่เป็นนามสกุล *.thl*

ไวยากรณ์ของ THL

การเยื้อง

การเยื้องหมายถึงช่องว่างที่อยู่ต้นบรรทัดของโค้ด

ในขณะที่ภาษาโปรแกรมอื่นๆ การเยื้องในโค้ดมีไว้เพื่อให้อ่านง่ายเท่านั้น แต่ใน THL การเยื้องมีความสำคัญมาก

THL ใช้การเยื้องเพื่อระบุบล็อกของโค้ด

ตัวอย่าง:

ถ้า $5 > 2$:

แสดง("5 มากกว่า 2 นะ")

ภาษา THL จะเกิดข้อผิดพลาดหาก你不ทำการเยื้อง

ตัวอย่าง: การเยื้องผิด

ถ้า $5 > 2$:

แสดง("5 มากกว่า 2 นะ")

จำนวนช่องว่างจะขึ้นอยู่กับคุณแล้วแต่จะกำหนด แต่โดยทั่วไปจะนิยมใช้ 4 ช่องว่างแต่ต้องมีอย่างน้อย 1 ช่อง

ตัวอย่าง :

ถ้า $5 > 2$:

แสดง("5 มากกว่า 2 นะ")

ถ้า $5 > 2$:

แสดง("5 มากกว่า 2 นะ")

แต่จำนวนช่องว่างต้องมีเท่ากันในแต่ละ บล็อกคำสั่ง ไม่งั้นภาษา THL จะให้ข้อผิดพลาดแก่คุณ

ตัวอย่าง: การเยื้องที่ไม่เท่ากัน

ถ้า $5 > 2$:

แสดง("5 มากกว่า 2 นะ")

แสดง("5 มากกว่า 2 นะ")

ตัวแปร

ในภาษา THL ตัวแปรจะถูกสร้างขึ้นก็ต่อเมื่อคุณกำหนดค่าให้มัน ซึ่งจะกล่าวโดยละเอียดในหัวข้อ ตัวแปร

ตัวอย่าง:

x คือ 5

y คือ "hello nsc"

การคอมเมนต์

ในภาษา THL การคอมเมนต์ทำการเขียนอธิบายคำสั่ง ในภาษา THL จะมีการคอมเมนต์สองประเภท

1 การคอมเมนต์แบบบรรทัดเดียว #

ตัวอย่าง:

แสดง("hellonsc") #เป็นการแสดงผลทางหน้าจอ

2 การคอมเมนต์แบบหลายบรรทัด /*.....*/

ตัวอย่าง:

แสดง("hellonsc")

/*เป็นการคอมเมนต์

แบบหลายบรรทัดก็ทำแบบนี้*/

การแสดงผล

การแสดงผลข้อความ

คุณสามารถใช้คำสั่ง **แสดง()** ในการแสดงค่าทางหน้าจอ

ตัวอย่าง:

```
แสดง("hellonsc")
```

คุณสามารถใช้ **แสดง()** ได้หลายครั้งโดยถ้าหากคุณต้องการเว้นบรรทัดต้องใส่ **\n** ต่อท้ายไม่งั้นจะไม่เกิดการเว้นบรรทัด

ตัวอย่าง:

```
แสดง("สวัสดี nsc\n")
```

```
แสดง("ฉันกำลังเรียน THL\n")
```

```
แสดง("โคตรเท่\n")
```

การใช้คำสั่ง **แสดง()** ข้อความที่ต้องการแสดงต้องอยู่ในเครื่องหมาย Double Quotess " " เท่านั้น ไม่งั้นภาษา THL จะแสดงข้อผิดพลาด

ตัวอย่าง: การใช้คำสั่งแสดงโดยไม่ใส่ Double Quotess

```
แสดง(สวัสดีnsc)
```

การแสดงตัวเลข

ในภาษา THL คุณก็สามารถแสดงตัวเลขได้เช่นกัน แต่ไม่เหมือนกับการแสดงข้อความ คือคุณไม่ต้องใส่เครื่องหมาย Double Quotess

ตัวอย่าง:

```
แสดง(44307)
```


นอกจากนี้คุณยังสามารถคิดเลขภายในคำสั่ง **แสดง** ได้อีกด้วย

ตัวอย่าง:

แสดง(3*4)

การผสมระหว่าง ข้อความกับ ตัวเลข

คุณสามารถแสดงข้อความกับตัวเลขรวมกัน โดยแยกด้วยเครื่องหมาย Comma

ตัวอย่าง:

แสดง("ฉันอายุ",17,"ปี")

ตัวแปร

ตัวแปร คือ ที่เก็บค่า

การสร้างตัวแปร

ในภาษา THL ไม่มีคำสั่งสำหรับการสร้างตัวแปร ตัวแปรจะถูกสร้างในตอนที่คุณ กำหนดค่าให้มัน

ตัวอย่าง

x คือ 5

y คือ "maxx"

แสดง(x, "\n")

แสดง(y)

note : ในภาษา THL ใช้คำว่า คือ ในการกำหนดค่า

ตัวแปรไม่จำเป็นต้องถูกประกาศด้วยชนิดข้อมูลที่เจาะจง นอกจากนี้ยังสามารถเปลี่ยนชนิดข้อมูลได้หลังจากที่ถูกกำหนดค่าไปแล้ว

ตัวอย่าง:

x คือ 5 # ชนิดของ x คือ จำนวนเต็ม

x คือ "maxx" # ชนิดของ x คือ ข้อความ

แสดง(x)

การเปลี่ยนชนิดข้อมูล

คุณสามารถเปลี่ยนชนิดข้อมูลได้ผ่านคำสั่ง **เปลี่ยนชนิดข้อมูล**

ตัวอย่าง :

x คือ 5 #x:จำนวนเต็ม

x คือ เปลี่ยนชนิดข้อมูล(x,ข้อความ) #x:ข้อความ

x คือ เปลี่ยนชนิดข้อมูล(x,ทศนิยม) #x:ทศนิยม

หากนิยาม เปลี่ยนชนิดข้อมูล(y,z) เมื่อ y คือ ชนิดข้อมูลเดิม , z คือชนิดข้อมูลที่สามารถเปลี่ยนได้ จะได้ตารางซึ่งบอกว่า ชนิดข้อมูลเดิมหนึ่งๆ สามารถเปลี่ยนเป็นชนิดข้อมูลอะไรได้บ้าง ดังตาราง

ชนิดข้อมูลเดิม (y)	ชนิดข้อมูลที่สามารถเปลี่ยนได้ (z)
จำนวนเต็ม	จำนวนเต็ม ข้อความ ค่าความจริง ทศนิยม
ข้อความ	จำนวนเต็ม ข้อความ ทศนิยม
ค่าความจริง	จำนวนเต็ม ค่าความจริง
ทศนิยม	จำนวนเต็ม ข้อความ ทศนิยม

Case-Sensitive

Case-Sensitive คือ การไวต่ออักษรพิมพ์ใหญ่-เล็ก

ตัวอย่าง :

a คือ "maxx"

A คือ "pabee"

A จะไม่เขียนทับ a

กฎสำหรับการตั้งชื่อตัวแปร:

- ชื่อตัวแปรต้องเริ่มต้นด้วยตัวอักษร (A-Z, a-z , ก-ฮ) หรือเครื่องหมายขีดล่าง (underscore _) เท่านั้น
- ชื่อตัวแปรห้ามเริ่มต้นด้วยตัวเลข
- ชื่อตัวแปรสามารถประกอบด้วยตัวอักษร ตัวเลข และเครื่องหมายขีดล่างเท่านั้น (A-z, 0-9, ก-ฮ, และ _)
- ชื่อตัวแปรไวต่อการพิมพ์ใหญ่-เล็ก (Case-sensitive) เช่น age, Age และ AGE จะถือว่าเป็นตัวแปรคนละตัวกัน
- ชื่อตัวแปรต้องไม่ตรงกับคำสงวน (Keywords) ของ THL

ตัวอย่าง:

myvar คือ "John"

my_var คือ "John"

_my_var คือ "John"

myVar คือ "John"

MYVAR คือ "John"

myvar2 คือ "John"

ตัวอย่าง: การตั้งชื่อที่ผิด

2myvar คือ "John"

my-var คือ "John"

my var คือ "John"

การแสดงผลตัวแปร

คุณสามารถใช้คำสั่ง **แสดง** ในการแสดงผลตัวแปรได้

ตัวอย่าง:

x คือ "THL โคตรเท"

แสดง(x)

ในคำสั่ง **แสดง** คุณสามารถแสดงหลายตัวแปรได้ โดยใช้เครื่องหมาย comma

ตัวอย่าง:

x คือ "THL "

y คือ "โคตร"

z คือ "เท"

แสดง(x, y, z)

หรืออาจใช้เครื่องหมาย + ในการแสดงหลายตัวแปรก็ได้

ตัวอย่าง:

x คือ "THL "

y คือ "โคตร"

z คือ "เห"

แสดง($x + y + z$)

สำหรับเครื่องหมาย + ยังสามารถใช้เป็นเครื่องหมายทางคณิตศาสตร์ได้ด้วย

ตัวอย่าง:

x คือ 3

y คือ 2

แสดง($x + y$)

ในคำสั่ง **แสดง** เวลาคุณพยายามรวมข้อความ กับตัวเลขด้วยเครื่องหมาย + ภาษา THL จะให้ข้อผิดพลาดแก่คุณ

ตัวอย่าง : รวมข้อความกับตัวเลขด้วย +

x คือ "maxx"

y คือ 5

แสดง($x + y$)

ดังนั้นวิธีที่ดีที่สุดในการแสดงหลายตัวแปร คือการใช้เครื่องหมาย comma ซึ่งจะรองรับหลายชนิดตัวแปร

ตัวอย่าง:

x คือ "maxx"

y คือ 5

แสดง(x, y)

ชนิดข้อมูล

ชนิดข้อมูลในภาษา THL ได้แก่

Text Type:	ข้อความ
Numeric Types:	จำนวนเต็ม , ทศนิยม
Sequence Types:	ชุดข้อมูล
Mapping Type:	คู่มือ-ค่า
Boolean Type:	ค่าความจริง
None Type:	ว่าง

ตัวดำเนินการทางคณิตศาสตร์

ตัวดำเนินการทางคณิตศาสตร์ใช้สำหรับการดำเนินการเกี่ยวกับตัวเลข

ตัวดำเนินการ	คำอธิบาย	ตัวอย่าง
+	บวกค่าสองค่าเข้าด้วยกัน	$x+y$
-	ลบค่าสองค่าออกด้วยกัน	$x-y$
*	คูณค่าสองค่าเข้าด้วยกัน	$x*y$
/	หารค่าสองค่า	x/y
%	หารเอาเศษ	x/y
//	หารเอาส่วน	$x//y$
**	ยกกำลัง	$x**y$
ราก	หารากที่ n	$x\text{ราก}y$
ln	หาค่าลอการิทึมธรรมชาติ	$\ln(x)$

ตัวอย่าง:

x คือ 15

y คือ 4

แสดง($x + y$,"n")

แสดง($x - y$,"n")

แสดง($x * y$,"n")

แสดง(x / y ,"n")

แสดง($x \% y$,"n")

แสดง($x ** y$,"n")

แสดง($x // y$,"n")

แสดง($x \text{ ราก } y$,"n")

แสดง($\ln(x)$)

คุณสามารถใช้วงเล็บเพื่อจัดลำดับความสำคัญของตัวดำเนินการได้

ตัวอย่าง:

แสดง(6ราก((1+2)*3/2%(9*6+4**5)//4) %1.5+((5+2)*(1.36+568)/2.35))

ตัวดำเนินการทางบิต

ตัวดำเนินการทางบิตใช้สำหรับการเปรียบเทียบบิต

ตัวดำเนินการ	ชื่อ	คำอธิบาย	ตัวอย่าง
&	AND	กำหนดค่าแต่ละบิตเป็น 1 หากทั้งสองบิตเป็น 1	$x \& y$
	OR	กำหนดค่าแต่ละบิตเป็น 1 หากบิตใดบิตหนึ่งใน สองบิตเป็น 1	$x y$
ซอร์	OXR	กำหนดค่าบิตแต่ละบิต เป็น 1 หากมีเพียงบิตใด บิตหนึ่งจากสองบิตเป็น 1	$x \text{ ซอร์ } y$
!	NOT	กลับค่าบิตทั้งหมด	$! x$
<<	Zero fill left shift	เลื่อนไปทางซ้ายโดยการ ดันเลขศูนย์จากทางขวา เข้าไป แล้วปล่อยให้บิต ซ้ายสุดหายไป	$x << y$
>>	Signed right shift	เลื่อนไปทางขวาโดยการ ดันสำเนาของบิตซ้ายสุด เข้ามาจากทางซ้าย และ ปล่อยให้บิตขวาสุดหลุด ออกไป	$x >> y$

ตัวอย่าง:

แสดง(9 & 6)

/*

9 ใน *binary* คือ 0000100 }
 6 ใน *binary* คือ 00000110 } ใช้หลักการ ทั้งสองอันเป็น 1 แล้วผลลัพธ์จะเป็น 1
 ดังนั้น 9 & 6 จึงเท่ากับ 00000000 ซึ่งก็คือ 0 ในระบบฐาน10

*/

แสดง(3 | 1)

/*

3 ใน *binary* คือ 00000011 } ใช้หลักการ มี1อันใดอันหนึ่งแล้วผลลัพธ์จะเป็น 1
1 ใน *binary* คือ 00000001 }

ดังนั้น 3 | 1 จึงเท่ากับ 00000011 ซึ่งก็คือ 3 ในระบบฐาน10

*/

แสดง(67 xor 69)

/*

67 ใน *binary* คือ 1000011 } ใช้หลักการ มีอันเดียวเท่านั้นที่เป็น 1 แล้วผลลัพธ์จะเป็น1
69 ใน *binary* คือ 1000101 }

ดังนั้น 67 | 69 จึงเท่ากับ 0000110 ซึ่งก็คือ 6 ในระบบฐาน10

*/

แสดง(!154)

/*

154 ใน binary คือ 10011010 ใช้หลักการสลับ 1 เป็น 0, 0 เป็น 1 ดังนั้น !154 คือ 01100101 (8บิต) ซึ่งก็คือ
-155 ในระบบเลขฐาน 10

*/

แสดง(3 << 2)

/*

3 ใน binary คือ 00000011 ใช้หลักการ ข้ายสุดตัดทิ้ง เพิ่ม 0 ทางขวา ในที่นี้คือ ทำ 2 ครั้งตามด้านขวา ของ
ตัวดำเนินการ

นั้นคือ

ครั้งที่ 0 : 00000011

ครั้งที่ 1 : 00000110

ครั้งที่ 2 : 00001100

ดังนั้น 3 << 2 คือ 00001100 ซึ่งก็คือ 12 ในระบบเลขฐาน 10 */

แสดง(8 >> 2)

/*

8 ใน binary คือ 00001000 ใช้หลักการ ตัดขวา เติมซ้าย ในที่นี้คือ ทำ 2 ครั้งตามด้านขวา
นั้นคือ

ครั้งที่ 0 : 00001000

ครั้งที่ 1 : 00000100

ครั้งที่ 2 : 00000010

ดังนั้น 3 << 2 คือ 00000010 ซึ่งก็คือ 2 ในระบบเลขฐาน 10

*/

ลำดับของตัวดำเนินการ

เมื่อนำตัวดำเนินการหลายตัวมาใช้รวมกัน ภาษาจะมีการ ลำดับความสำคัญแสดงไว้ในตารางด้านล่าง โดยเริ่มจาก ลำดับความสำคัญสูงสุดไว้ด้านบน แต่หากตัวดำเนินการสองตัวมีลำดับความสำคัญเท่ากัน นิพจน์ จะถูกประเมิน จากซ้ายไปขวา

()
ไม่ , !
ln
** , ราก
* , / , // , %
+ , -
<< , >>
> , < , >= , <=
= , !=
&
ชอร์

ชุดข้อมูล

การสร้างชุดข้อมูล

ชุดข้อมูล ใช้ในการเก็บหลายข้อมูลในตัวแปรเดียว

ชุดข้อมูลสามารถสร้างได้โดยใช้ square brackets

ตัวอย่าง:

ของชอบ คือ ["กล้วยเดี่ยว","รถยนต์","เพลง","เธอ"]

แสดง(ของชอบ)

ชุดข้อมูลสามารถมีหลายชนิดตัวแปร หรือแม้กระทั่งตัวแปรใน ชุดข้อมูลเดียวกัน

ตัวอย่าง:

x คือ "ฉัน"

y คือ "และ"

รวมๆ คือ ["สวัสดิ์" , 20 ,3.14 ,จริง,[1,2,3,4] ,x,y,"เธอ"]

แสดง(รวมๆ)

การเข้าถึงสมาชิกในชุดข้อมูล

การเข้าถึงสมาชิกในชุดข้อมูลจะใช้เลข index ในการเข้าถึง โดยสมาชิกตัวแรกจะใช้เลข index ที่ 0 ตัวที่สองจะใช้เป็น 1 เรียงกันไปเรื่อยๆ

เช่น หาก ชุดข้อมูล คือ ["maxx","phawat","racha","com101"] ลำดับของ index จะเป็น

"maxx"	"phawat"	"racha"	"com101"
0	1	2	3

ตัวอย่าง :

x คือ ["maxx","phawat","racha","com101"]

แสดง(x[0],"\\n",x[2])

นอกจากนี้ยังสามารถเข้าถึงชุดข้อมูลที่ซับซ้อนได้อีกด้วย

ตัวอย่าง:

x คือ [[[1, 2]], [[3, 4]]]

แสดง(x[0][0][1])

การเปลี่ยนข้อมูลในชุดข้อมูล

สำหรับการเปลี่ยนข้อมูลในชุดข้อมูลต้องอ้างอิงเลข index

ตัวอย่าง:

ผลไม้ คือ ["ส้ม","กล้วย","องุ่น"]

ผลไม้[0] คือ "แอปเปิล"

แสดง(ผลไม้)

การนำชุดข้อมูลมาต่อกัน

คุณสามารถนำชุดข้อมูลมาต่อกันโดยใช้เครื่องหมาย +

ตัวอย่าง:

list1 คือ ["a", "b", "c"]

list2 คือ [1, 2, 3]

list3 คือ list1 + list2

แสดง(list3)

คำสั่งที่เกี่ยวข้องกับชุดข้อมูล

ตัวอย่าง list คือ [1,2,3,4,5]

คำสั่ง	คำอธิบาย	โค้ด
.ขนาด()	จะคืนค่าเป็นขนาดของชุดข้อมูล	แสดง(list.ขนาด())
.เพิ่ม()	จะเพิ่มข้อมูลใหม่ให้กับชุดข้อมูล	list.เพิ่ม(6) แสดง(list)
.ดึงออก()	เป็นการดึงข้อมูลตัวสุดท้ายออกจากชุดข้อมูล	list.ดึงออก() แสดง(list)
.แทรก()	เป็นการแทรกข้อมูลไปยัง index ต่างๆ	list.แทรก(2,20) แสดง(list)
.ลบ()	เป็นการลบข้อมูลใน index ต่างๆ	list. ลบ(3) แสดง(list)

คู่มือ-ค่า

การสร้างคู่มือ-ค่า

คู่มือ-ค่า คือการเก็บข้อมูลโดยใช้ คีย์เพื่อเข้าถึงค่า

ตัวอย่าง:

ตัวฉัน คือ {

"ชื่อ" : "ภวดี",

"อายุ" : 17,

"สิ่งที่ชอบ" : ["กล้วยเตียว","รถยนต์","เพลง","เธอ"],

"สัตว์เลี้ยง" : {

"ชื่อ" : "ดำ",

"อายุ" : 2,

"สีขน" : ["ดำ","เหลือง"]

}

}

แสดง(ตัวฉัน)

การเข้าถึงค่าในคู่มือ-ค่า

คุณสามารถเข้าถึงค่าในคู่มือ-ค่า ได้ผ่านการอ้างอิงผ่านคีย์

ตัวอย่าง:

ตัวฉัน คือ {

"ชื่อ" : "ภวดี",

"อายุ" : 17,

}

แสดง(ตัวฉัน.ชื่อ)

นอกจากนี้คุณก็ยังสามารถเข้าถึงค่าที่ซับซ้อนได้ด้วย

ตัวอย่าง:

ตัวฉัน คือ {

 "ชื่อ" : "ภวัต",

 "อายุ" : 17,

 "สิ่งที่ชอบ" : ["มวยเดี่ยว","รถยนต์","เพลง","เธอ"],

 "สัตว์เลี้ยง" : {

 "ชื่อ" : "ดำ",

 "อายุ" : 2,

 "สีขน" : ["ดำ","เหลือง"]

 }

}

แสดง(ตัวฉัน.สัตว์เลี้ยง.สีขน[0])

การเปลี่ยนค่า ในคูคีย์-ค่า

คุณสามารถเปลี่ยนค่าในคูคีย์-ค่าโดยการอ้างอิงผ่านคีย์

ตัวอย่าง:

ตัวฉัน คือ {

 "ชื่อ" : "ภวัต",

 "อายุ" : 17,

}

ตัวฉัน.ชื่อ คือ "ชวกร"

แสดง(ตัวฉัน.ชื่อ)

เครื่องหมายเปรียบเทียบ

ในภาษา THL รอบรับเครื่องหมายเปรียบเทียบทางคณิตศาสตร์ต่อไปนี้

- เท่ากับ: $a = b$
- ไม่เท่ากับ: $a \neq b$
- น้อยกว่า: $a < b$
- น้อยกว่าหรือเท่ากับ: $a \leq b$
- มากกว่า: $a > b$

มากกว่าหรือเท่ากับ: $a \geq b$

note : ในภาษา THL เครื่องหมายเท่ากับใช้ = แค่อันเดียว

โดยเครื่องหมายเหล่านี้สามารถนำไปใช้ได้หลายทาง แต่ส่วนมากจะใช้ในเงื่อนไข และการทำซ้ำ

เงื่อนไข “ถ้า”

การใช้งานคำสั่ง ถ้า

คำสั่ง **ถ้า** จะตรวจสอบเงื่อนไข (ซึ่งเป็นนิพจน์ที่ให้ผลลัพธ์เป็นจริงหรือเท็จ) ถ้าเงื่อนไขนั้นเป็นจริง โค้ดบล็อกภายในคำสั่ง **ถ้า** จะทำงาน แต่ถ้าเงื่อนไขเป็นเท็จ โค้ดบล็อกนั้นจะถูกข้ามไป

ตัวอย่าง:

a คือ 900

b คือ 30

ถ้า $a > b$:

แสดง("a มากกว่า b")

ในตัวอย่างนี้ เราใช้ตัวแปรสองตัวคือ **a** และ **b** ซึ่งถูกนำมาใช้เป็นส่วนหนึ่งของคำสั่ง **ถ้า** เพื่อทดสอบว่า **a** มีค่ามากกว่า **b** หรือไม่ และเนื่องจาก **a** มีค่าเท่ากับ 900 และ **b** มีค่าเท่ากับ 30 เรารู้ว่า 900 มีค่ามากกว่า 30 ดังนั้นคำสั่ง **แสดง("a มากกว่า b")** จึงทำงาน

การเยื้องในคำสั่งถ้า

ภาษาโปรแกรม THL ใช้การเยื้อง (การเว้นวรรคที่ต้นบรรทัด) ในการกำหนดขอบเขต ของโค้ด ในขณะที่ภาษาโปรแกรมอื่นๆ มักจะใช้เครื่องหมายปีกกาเพื่อจุดประสงค์นี้

ตัวอย่าง : หากไม่มีการเยื้องจะเกิดข้อผิดพลาด

a คือ 900

b คือ 30

ถ้า $a > b$:

แสดง("a มากกว่า b")

การเขียนหลายคำสั่งใน ถ้า

คุณสามารถใส่คำสั่งหลายๆ คำสั่งไว้ภายในบล็อก **ถ้า** ได้ โดยทุกคำสั่งเหล่านั้นจะต้องเขียนเยื้องให้อยู่ในระดับเดียวกันทั้งหมด

ตัวอย่าง:

ความรักที่คุณต้องการ คือ 60

ความรักที่ผมให้คุณ คือ 70

ถ้า ความรักที่ผมให้คุณ $\geq$ ความรักที่คุณต้องการ:

 แสดง("คุณประทับใจ\n")

 แสดง("ผมมีความสุข\n")

 แสดง("เราเป็นแฟนกัน")

การใช้งานตัวแปรในเงื่อนไข

คุณสามารถนำค่าความจริง ไปใช้ในเงื่อนไขของ **ถ้า** ได้โดยตรงเลย โดยไม่จำเป็นต้องใช้ตัวดำเนินการเปรียบเทียบ
อีก

ตัวอย่าง:

is_logged_in คือ จริง

ถ้า is_logged_in:

 แสดง("Welcome back!")

เงื่อนไข “มิฉะนั้นถ้า”

การใช้งานคำสั่ง มิฉะนั้นถ้า

คำว่า **มิฉะนั้นถ้า** เป็นวิธีของภาษา THL ในการบอกว่า "หากเงื่อนไขก่อนหน้านี้ทั้งหมดเป็นเท็จ ให้ลองมาตรวจสอบเงื่อนไขนี้ดู"

คำสั่ง **มิฉะนั้นถ้า** ช่วยให้คุณสามารถตรวจสอบเงื่อนไขได้หลากหลายรูปแบบ และจะเริ่มทำงานในบล็อกโค้ดทันทีที่มีเงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง

ตัวอย่าง:

a คือ 33

b คือ 33

ถ้า $b > a$:

แสดง("b มากกว่า a")

มิฉะนั้นถ้า $a = b$:

แสดง("a และ b เท่ากัน")

ในตัวอย่างนี้ **a** มีค่าเท่ากับ **b** ดังนั้นเงื่อนไขแรกจึงเป็นเท็จ แต่เงื่อนไขของ **มิฉะนั้นถ้า** นั้นเป็นจริงเราจึงแสดงผลบนหน้าจอว่า "a และ b เท่ากัน"

การใช้งานคำสั่ง มิฉะนั้นถ้า หลายครั้ง

คุณสามารถใส่คำสั่ง มิฉะนั้นถ้า ก็คำสั่งก็ได้ตามที่ต้องการ โดย THL จะตรวจสอบแต่ละเงื่อนไขเรียงตามลำดับ และจะทำงานในบล็อกโค้ดของเงื่อนไขแรกที่เป็นจริงทันที

ตัวอย่าง:

คะแนน คือ 75

ถ้า คะแนน ≥ 90 :

แสดง("เกรด: A")

มิฉะนั้นถ้า คะแนน ≥ 80 :

```
แสดง("เกรด: B")
มิฉะนั้นถ้า คะแนน >= 70:
    แสดง("เกรด: C")
มิฉะนั้นถ้า คะแนน >= 60:
    แสดง("เกรด: D")
```

ในตัวอย่างนี้ โปรแกรมจะตรวจสอบแต่ละเงื่อนไขเรียงตามลำดับ เนื่องจากคะแนน คือ 75 โปรแกรมจึงแสดงผลคำว่า "เกรด: C" ออกมา (ซึ่งเป็นเงื่อนไขแรกที่มีค่าเป็นจริง)

คำสั่ง มิฉะนั้นถ้า ทำงานอย่างไร?

เมื่อคุณใช้ **มิฉะนั้นถ้า** ภาษา THL จะตรวจสอบและประเมินเงื่อนไขจากบนลงล่าง ทันทันทีที่มันเจอเงื่อนไขที่เป็นจริง มันจะทำงานในบล็อกโค้ดนั้นทันที และจะข้ามเงื่อนไขที่เหลือทั้งหมดที่ต่อท้ายมา

ข้อสำคัญ: เฉพาะเงื่อนไขแรกที่เป็นจริงเท่านั้นที่จะถูกประมวลผล ถึงแม้ว่าจะมีหลายเงื่อนไขที่เป็นจริงพร้อมกัน แต่ THL จะหยุดทำงานทันทีหลังจากที่ได้รับบล็อกโค้ดแรกที่ตรงเงื่อนไขไปแล้ว

```
ตัวอย่าง:
อายุ คือ 25

ถ้า อายุ < 13:
    แสดง("คุณคือเด็ก")
มิฉะนั้นถ้า อายุ < 20:
    แสดง("คุณคือวัยรุ่น")
มิฉะนั้นถ้า อายุ < 65:
    แสดง("คุณคือผู้ใหญ่")
มิฉะนั้นถ้า อายุ >= 65:
    แสดง("คุณคือคนแก่")
```

เมื่อไหร่จะใช้ มิฉะนั้นถ้า?

ให้ใช้ **มิฉะนั้นถ้า** เมื่อคุณมีหลายเงื่อนไขที่เป็นแบบ แยกต่างหากจากกันโดยสิ้นเชิง (Mutually Exclusive) ที่ต้องตรวจสอบ วิธีนี้จะมีประสิทธิภาพ มากกว่าการใช้คำสั่ง ถ้า แยกกันหลายๆ ตัว เพราะ THL จะหยุดตรวจสอบเงื่อนไขทันทีที่มันเจอเงื่อนไขแรกที่เป็นจริง

ตัวอย่าง:

วัน คือ 3

ถ้า วัน = 1:

แสดง("วันจันทร์")

มิฉะนั้นถ้า วัน = 2:

แสดง("วันอังคาร")

มิฉะนั้นถ้า วัน = 3:

แสดง("วันพุธ")

มิฉะนั้นถ้า วัน = 4:

แสดง("วันพฤหัสบดี")

มิฉะนั้นถ้า วัน = 5:

แสดง("วันศุกร์")

มิฉะนั้นถ้า วัน = 6:

แสดง("วันเสาร์")

มิฉะนั้นถ้า วัน = 7:

แสดง("วันอาทิตย์")

เงื่อนไข “มีฉะนั้น”

การใช้งานคำสั่ง มีฉะนั้น

คำสั่ง **มีฉะนั้น** จะคอยดักจับทุกกรณีที่ไม่ได้ถูกดักจับโดยเงื่อนไขก่อนหน้านี้ทั้งหมด

คำสั่ง **มีฉะนั้น** จะถูกประมวลผล ก็ต่อเมื่อเงื่อนไขของ **ถ้า** (รวมถึงเงื่อนไขของ **มีฉะนั้นถ้า** ทั้งหมด ถ้ามี) มีค่าเป็นเท็จ

ตัวอย่าง:

a คือ 200

b คือ 33

ถ้า $b > a$:

 แสดง("b มากกว่า a")

มีฉะนั้นถ้า $a = b$:

 แสดง("a และ b มีค่าเท่ากัน")

มีฉะนั้น:

 แสดง("a มากกว่า b")

ในตัวอย่างนี้ **a** มีค่ามากกว่า **b** ดังนั้นเงื่อนไขแรกจึงเป็นเท็จ และเงื่อนไขของ มีฉะนั้นถ้า ก็ไม่เป็นจริงด้วยเช่นกัน เราจึงข้ามมาที่เงื่อนไขของ มีฉะนั้น แล้วแสดงผลบนหน้าจอว่า "a มากกว่า b"

การใช้ มีฉะนั้น โดยไม่มี มีฉะนั้นถ้า

คุณสามารถใช้ **มีฉะนั้น** โดยที่ไม่มี **มีฉะนั้นถ้า** ก็ได้

ตัวอย่าง:

a คือ 200

b คือ 33

ถ้า $b > a$:

 แสดง("b มากกว่า a")

มีฉะนั้น:

 แสดง("b ไม่มากกว่า a")

มีฉะนั้น ทำงานอย่างไร?

คำสั่ง **มีฉะนั้น** ทำหน้าที่กำหนดสิ่งที่จะให้โปรแกรมทำเป็นค่าเริ่มต้น (Default Action) ในกรณีที่ไม่มีเงื่อนไขใดๆ ก่อนหน้านี้เป็นจริงเลย ให้ลองคิดว่ามันเป็น "ตัวดักจับทุกอย่าง" สำหรับทุกสถานการณ์ที่ไม่ได้ครอบคลุมอยู่ในคำสั่ง **ถ้า** และ **มีฉะนั้นถ้า** ของคุณ

หมายเหตุ: คำสั่ง **มีฉะนั้น** จะต้องอยู่ท้ายสุดเสมอ คุณไม่สามารถมี **มีฉะนั้นถ้า** ตามหลัง **มีฉะนั้น** ได้

ตัวอย่าง:

ตัวเลข คือ 7

ถ้า ตัวเลข % 2 = 0:

แสดง("ตัวเลขนั้นเป็นเลขคู่")

มีฉะนั้น:

แสดง("ตัวเลขนั้นเป็นเลขคี่")

ตัวดำเนินการทางตรรกศาสตร์

ตัวดำเนินการทางตรรกศาสตร์ของTHL

ตัวดำเนินการทางตรรกศาสตร์ ถูกนำมาใช้เพื่อรวมประโยคเงื่อนไขเข้าด้วยกัน โดยในภาษา THL จะมีตัวดำเนินการทางตรรกศาสตร์อยู่ 3 ตัว ได้แก่:

- **และ** — จะคืนค่าเป็นจริง ก็ต่อเมื่อประโยคเงื่อนไขทั้งสองเป็นจริง
- **หรือ** — จะคืนค่าเป็นจริง ขอแค่มีประโยคเงื่อนไขใดเงื่อนไขหนึ่งเป็นจริง
- **ไม่** — จะกลับผลลัพธ์ให้เป็นตรงกันข้าม โดยจะคืนค่าเป็นเท็จ หากผลลัพธ์เดิมเป็นจริง

ตัวดำเนินการ และ

และ เป็นตัวดำเนินการทางตรรกศาสตร์ ซึ่งถูกนำมาใช้เพื่อรวมประโยคเงื่อนไขเข้าด้วยกัน โดยเงื่อนไขทั้งสองจะต้องเป็นจริง เท่านั้น จึงจะทำให้พจน์ ทั้งหมดมีค่าเป็นจริง

ตัวอย่าง:

a คือ 200

b คือ 33

c คือ 500

ถ้า $a > b$ และ $c > a$:

แสดง("ทั้งสองเงื่อนไขเป็นจริง")

ตัวดำเนินการ หรือ

หรือ เป็นตัวดำเนินการทางตรรกศาสตร์ซึ่ง ถูกนำมาใช้เพื่อรวมประโยคเงื่อนไขเข้าด้วยกัน โดยขอแค่มีอย่างน้อยหนึ่งเงื่อนไขที่เป็นจริง ก็จะทำให้พจน์ทั้งหมดมีค่าเป็นจริง

ตัวอย่าง:

a คือ 200

b คือ 33

c คือ 500

ถ้า $a > b$ หรือ $c > a$:

แสดง("เงื่อนไขอย่างน้อยอันใดอันหนึ่งเป็นจริง")

ตัวดำเนินการ ไม่

ไม่ เป็นตัวดำเนินการทางตรรกศาสตร์ ซึ่งถูกนำมาใช้เพื่อกลับผลลัพธ์ของประโยคเงื่อนไขให้เป็นตรงกันข้าม

ตัวอย่าง:

a คือ 33

b คือ 200

ถ้า ไม่ ($a > b$):

แสดง("a ไม่ได้มากกว่า b")

การรวมกันของหลายตัวดำเนินการ

คุณสามารถผสมตัวดำเนินการทางตรรกศาสตร์หลายๆ ตัวเข้าด้วยกันในพจน์ เดียวได้ โดย THL จะเรียงลำดับการประมวลผลจาก **ไม่** ก่อนเป็นอันดับแรก ตามด้วย **และ** และปิดท้ายด้วย **หรือ**

ตัวอย่าง:

อายุ คือ 25

เป็นนักเรียน คือ เท็จ

มีโค้ดส่วนลด คือ จริง

ถ้า (อายุ < 18 หรือ อายุ > 65) และ ไม่ เป็นนักเรียน หรือ มีโค้ดส่วนลด:

แสดง("ได้ส่วนลด!")

การใช้เครื่องหมายวงเล็บเพื่อความชัดเจน

เมื่อต้องผสมตัวดำเนินการทางตรรกศาสตร์หลายๆ ตัวเข้าด้วยกัน ให้ใช้เครื่องหมายวงเล็บเพื่อระบุเจตนาของคุณให้ชัดเจน และเพื่อควบคุมลำดับการประมวลผล เงื่อนไขเหล่านั้น

ตัวอย่าง:

ฉันมาโรงเรียน คือ จริง

เธอมาโรงเรียน คือ จริง

วันนี้โรงเรียนหยุด คือ จริง

ถ้า (ฉันมาโรงเรียน และ ไม่ เธอมาโรงเรียน) หรือ วันนี้โรงเรียนหยุด:
แสดง("วันนี้เป็นวันที่ดี")

ตัวอย่างเพิ่มเติม

ตัวอย่าง:

ชื่อผู้ใช้ คือ "maxxtebza007"

รหัสผ่าน คือ "Racha1101"

ผ่านการยืนยันตัวตน คือ จริง

ถ้า ชื่อผู้ใช้ = "maxxtebza007" และ รหัสผ่าน = "Racha1101" และ ผ่านการยืนยันตัวตน:
แสดง("การล็อกอินสำเร็จ")

มิฉะนั้น:

แสดง("การล็อกอินไม่สำเร็จ")

เงื่อนไขซ้อนเงื่อนไข

การใช้งาน เงื่อนไขซ้อนเงื่อนไข

ตัวอย่าง:

คะแนน คือ 85

การเข้าชั้นเรียน คือ 90

การส่งงาน คือ จริง

ถ้า คะแนน ≥ 60 :

 ถ้า การเข้าชั้นเรียน ≥ 80 :

 ถ้า การส่งงาน:

 แสดง("ผ่านและส่งงานครบ")

 มิฉะนั้น:

 แสดง("ผ่านแต่ส่งงานไม่ครบ")

 มิฉะนั้น:

 แสดง("ผ่านแต่ไม่ค่อยเข้าชั้นเรียน")

มิฉะนั้น:

 แสดง("ตก")

คำสั่ง ขณะ

การใช้คำสั่ง ขณะ

ด้วยคำสั่ง **ขณะ** เราจะสามารถสั่งให้กลุ่มคำสั่งทำงานซ้ำไปได้เรื่อยๆ ตราบใดที่เงื่อนไขยังคงเป็นจริง

ตัวอย่าง:

i คือ 1

ขณะ $i < 6$:

 แสดง(i)

 i คือ $i + 1$

คำสั่ง หยุด

ด้วยคำสั่ง **หยุด** เราจะสามารถสั่งให้ลูบหยุดทำงานได้ทันที แม้ว่าเงื่อนไขของ ขณะ จะยังคงเป็นจริงอยู่ก็ตาม

ตัวอย่าง:

i คือ 1

ขณะ $i < 6$:

 แสดง(i)

 ถ้า $i = 3$:

 หยุด

 i คือ $i + 1$

คำสั่ง ถัดไป

ด้วยคำสั่ง **ถัดไป** เราจะสามารถสั่งให้หยุดการทำงานรอบปัจจุบัน แล้วข้ามไปทำงานในรอบถัดไปได้ทันที

ตัวอย่าง:

i คือ 0

ขณะ $i < 6$:

i คือ $i + 1$

ถ้า $i = 3$:

ถัดไป

แสดง(i)

คำสั่ง สำหรับ

คำสั่งสำหรับ ใช้ในการวนซ้ำโดยที่เรารู้จำนวนการวนซ้ำที่แน่นอน

คำสั่ง “สำหรับ” แบบที่ผ่านค่าตัวเดียว

คำสั่งแบบนี้จะทำการวนซ้ำจาก 0 ไปยังก่อนตัวเลขที่กำหนดโดยจะเพิ่มค่าทีละ 1

ตัวอย่าง:

สำหรับ i ในช่วง(6):

แสดง(i)

คำสั่ง “สำหรับ” แบบที่ผ่านค่าสองค่า

คำสั่งแบบนี้จะทำให้เราสามารถกำหนดช่วงของการวนซ้ำที่ต้องการได้โดยที่ค่าจะยังคงเพิ่มทีละ 1

ตัวอย่าง:

สำหรับ i ในช่วง(10 , 20):

แสดง(i, " ")

คำสั่ง “สำหรับ” แบบที่ผ่านค่าสามค่า

คำสั่งแบบนี้จะทำให้เราสามารถกำหนดช่วงของการวนซ้ำ และกำหนดการเพิ่มค่าของตัวแปรได้

ตัวอย่าง:

สำหรับ i ในช่วง(10,20,2):

แสดง(i, " ")

ฟังก์ชัน

ความหมายของฟังก์ชัน

ฟังก์ชัน ใน THL คือการรวมกลุ่มของคำสั่งที่ทำงานเฉพาะอย่างใดอย่างหนึ่งมารวมไว้ด้วยกัน เพื่อให้เราสามารถเรียกใช้งานซ้ำๆ ได้โดยไม่ต้องเขียนโค้ดเดิมใหม่ทุกครั้ง ช่วยให้โค้ดสะอาด อ่านง่าย และดูแลรักษาง่ายขึ้น

การสร้างฟังก์ชัน

ในภาษา THL ฟังก์ชันจะถูกกำหนดขึ้นโดยการใช้ คำว่า **โปรแกรม** แล้วตามด้วยชื่อฟังก์ชันและเครื่องหมายวงเล็บ

ตัวอย่าง:

โปรแกรม ฟังก์ชันฉันของฉัน():

 แสดง("สวัสดีฉันคือฟังก์ชัน")

สิ่งนี้จะเป็นการสร้างฟังก์ชันที่มีชื่อว่า ฟังก์ชันฉันของฉัน ซึ่งจะแสดงผลข้อความ ห"สวัสดีฉันคือฟังก์ชัน" เมื่อถูกเรียกใช้งาน

โค้ดที่อยู่ภายในฟังก์ชันจะต้องถูกเียง โดยภาษา THL จะใช้การเียงเพื่อกำหนดขอบเขตของบล็อกโค้ด

การเรียกฟังก์ชัน

การเรียกฟังก์ชันทำได้โดยการเขียนชื่อฟังก์ชันและตามด้วยวงเล็บ

ตัวอย่าง:

โปรแกรม ฟังก์ชันฉันของฉัน():

 แสดง("สวัสดีฉันคือฟังก์ชัน")

ฟังก์ชันฉันของฉัน()

คุณสามารถเรียกฟังก์ชันหลายครั้งได้

ตัวอย่าง:

โปรแกรม ฟังก์ชันฉันของฉัน():

 แสดง("สวัสดิ์ฉันคือฟังก์ชัน\n")

ฟังก์ชันฉันของฉัน()

ฟังก์ชันฉันของฉัน()

ฟังก์ชันฉันของฉัน()

ทำไมต้องใช้ฟังก์ชัน

ลองจินตนาการว่า คุณจำเป็นต้องแปลงอุณหภูมิจากองศาฟาเรนไฮต์เป็นเซลเซียสตั้งหลายครั้งในโปรแกรมของคุณ ถ้าไม่มีฟังก์ชัน คุณก็จะต้องเขียนโค้ดสำหรับคำนวณแบบเดิมซ้ำแล้วซ้ำเล่า

ตัวอย่าง:

ฟาเรนไฮต์1 คือ 77

เซลเซียส1 คือ (ฟาเรนไฮต์1 - 32) * 5 / 9

แสดง(เซลเซียส1,"\n")

ฟาเรนไฮต์2 คือ 95

เซลเซียส2 คือ (ฟาเรนไฮต์2 - 32) * 5 / 9

แสดง(เซลเซียส2,"\n")

ฟาเรนไฮต์3 คือ 50

เซลเซียส3 คือ (ฟาเรนไฮต์3 - 32) * 5 / 9

แสดง(เซลเซียส3,"\n")

พอมิฟังก์ชันเข้ามาช่วย คุณก็แค่เขียนโค้ดนั้นเพียงครั้งเดียว แล้วสามารถนำกลับมาใช้ซ้ำได้เลย

ตัวอย่าง:

โปรแกรม ฟาเรนไฮต์เป็นเซลเซียส(ฟาเรนไฮต์):

คีนค่า (ฟาเรนไฮต์ - 32) * 5 / 9

แสดง(ฟาเรนไฮต์เป็นเซลเซียส(77),"\\n")

แสดง(ฟาเรนไฮต์เป็นเซลเซียส(95),"\\n")

แสดง(ฟาเรนไฮต์เป็นเซลเซียส(50),"\\n")

อาร์กิวเมนต์

เราสามารถส่งข้อมูลเข้าไปในฟังก์ชันได้ในรูปแบบของ อาร์กิวเมนต์

โดยอาร์กิวเมนต์จะถูกระบุไว้ในเครื่องหมายวงเล็บที่อยู่ต่อท้ายชื่อฟังก์ชัน คุณจะใส่กี่อาร์กิวเมนต์ก็ได้ตามที่ต้องการ เพียงแค่ใช้เครื่องหมายจุลภาค คั่นระหว่างแต่ละอาร์กิวเมนต์

ตัวอย่างต่อไปนี้จะป็นฟังก์ชันที่มีหนึ่งอาร์กิวเมนต์ (ชื่อว่า ชื่อต้น) ซึ่งเมื่อฟังก์ชันนี้ถูกเรียกใช้งาน เราจะส่งค่าชื่อจริงเข้าไป และฟังก์ชันจะนำค่านั้นไปใช้เพื่อแสดงผลเป็นชื่อเต็มออกมา

ตัวอย่าง:

โปรแกรม ต่อชื่อ(ชื่อต้น):

แสดง(ชื่อต้น + " มติธรรม\\n")

ต่อชื่อ("ภาวัต")

ต่อชื่อ("ชวกร")

ต่อชื่อ("แผ่นฟ้า")

ในภาษา THL คุณสามารถมีอาร์กิวเมนต์หลายค่าได้โดยแยกด้วยเครื่องหมายจุลภาค

ตัวอย่าง:

โปรแกรม บวก(ก,ข,ค):

 แสดง(ก+ข+ค)

บวก(3,2,4)

แต่จำนวนอาร์กิวเมนต์ต้องตรงกับจำนวนพารามิเตอร์ไม่งั้นภาษา THL จะให้ข้อผิดพลาดแก่คุณ

ตัวอย่าง:จำนวนอาร์กิวเมนต์ไม่ตรงกับพารามิเตอร์

โปรแกรม บวก(ก,ข,ค):

 แสดง(ก+ข+ค)

บวก(3,2)

พารามิเตอร์ต่างจากอาร์กิวเมนต์อย่างไร?

คำว่า พารามิเตอร์ และ อาร์กิวเมนต์ มักจะถูกใช้ในความหมายเดียวกัน นั่นคือ ข้อมูลที่ถูกส่งเข้าไปในฟังก์ชัน

แต่ถ้ามองจากมุมมองของฟังก์ชันแล้ว ทั้งสองคำมีความแตกต่างกันเล็กน้อย ดังนี้

- พารามิเตอร์: คือ ตัวแปรที่ถูกเขียนไว้ในวงเล็บตอนที่เรานิยามหรือสร้างฟังก์ชัน
- อาร์กิวเมนต์: คือ ค่าจริง ๆ ที่เราส่งเข้าไปให้ฟังก์ชันตอนที่เราเรียกใช้งานมัน

ตัวอย่าง:

โปรแกรม ต่อชื่อ(ชื่อต้น):# ชื่อต้น คือ พารามิเตอร์

 แสดง(ชื่อต้น + " มติธรรม\n")

ต่อชื่อ("กวัด") # "กวัด" คือ อาร์กิวเมนต์

พารามิเตอร์ตั้งต้น

คุณสามารถกำหนดค่าเริ่มต้น ให้กับพารามิเตอร์ได้ ซึ่งถ้าฟังก์ชันถูกเรียกใช้งานโดยไม่มีการส่งอาร์กิวเมนต์ใด ๆ เข้าไป มันก็จะดึงค่าเริ่มต้นนี้มาใช้งานแทน

ตัวอย่าง:

โปรแกรม แนะนำตัว(ประเทศ คือ "ไทย"):

แสดง("ฉันมาจากประเทศ ", ประเทศ,"\n")

แนะนำตัว("ลาว")

แนะนำตัว("อังกฤษ")

แนะนำตัว()

แนะนำตัว("พม่า")

การคืนค่า

ฟังก์ชันสามารถส่งข้อมูลกลับไปยังโค้ดที่เรียกใช้งานมันได้ โดยการใช้คำสั่ง **คืนค่า** และเมื่อฟังก์ชันทำงานมาถึงคำสั่ง **คืนค่า** มันจะหยุดประมวลผลทันที พร้อมกับส่งผลลัพธ์นั้นกลับไป

ตัวอย่าง:

โปรแกรม เริ่มทักทาย():

คืนค่า "สวัสดีเธอชื่ออะไร"

ทักทาย คือ เริ่มทักทาย()

แสดง(ทักทาย)

คุณสามารถนำค่าที่ส่งกลับมา ไปใช้งานต่อได้โดยตรงเลย

ตัวอย่าง:

โปรแกรม เริ่มทักทาย():

คืนค่า "สวัสดีเธอชื่ออะไร"

แสดง(เริ่มทักทาย())

คุณสามารถใช้การคืนค่ารวมกับอาร์กิวเมนต์ได้

ตัวอย่าง:

โปรแกรม บวก(ก,ข):

คืนค่า ก+ข

คำตอบ คือ บวก(5,2)

แสดง(คำตอบ)

โมดูล

ความหมายของโมดูล

ให้คิดซะว่า โมดูลก็เหมือนกับคลังโค้ด สำเร็จรูปชิ้นหนึ่ง

มันคือไฟล์ที่รวบรวมกลุ่มของฟังก์ชันต่าง ๆ เอาไว้ด้วยกัน เพื่อให้คุณสามารถดึงไปใช้งานร่วมกับแอปพลิเคชันหรือโปรแกรมของคุณได้

การสร้างโมดูล

ให้คุณสร้างไฟล์นามสกุล **.thl** แล้วสร้างฟังก์ชัน แต่ให้ใส่คำว่าส่งออก ก่อนคำว่าโปรแกรมดังในตัวอย่างสร้างฟังก์ชันในไฟล์ชื่อ **add.thl**

ตัวอย่าง:

ส่งออก โปรแกรม บวก(a,b):

แสดง(a+b)

จากนั้นให้พิมพ์คำสั่ง **mmt** ตามด้วยชื่อไฟล์ที่เป็น **.thl** จากนั้นต่อด้วย ชื่อไฟล์ที่ต้องการที่เป็น **.json** ระบบจะสร้างไฟล์ **JSON** ขึ้นมา

ตัวอย่าง:

```
mmt add.thl addnum.json
```

เพียงแค่นี้คุณก็ได้ทำการสร้างโมดูลเสร็จสิ้นแล้ว

การใช้งานโมดูล

คุณสามารถใช้งานโมดูลที่สร้างขึ้นได้ผ่านการใช้คำสั่ง **นำเข้า** ตามด้วยไฟล์ JSON ที่ได้สร้างขึ้น

ตัวอย่าง:

นำเข้า "addnum.json"

บวก(2,3)

นอกจากนี้คุณยังสามารถนำเข้าโดยการเขียน path ของไฟล์ได้ด้วย

ตัวอย่าง:

นำเข้า "D:\\thl\\addnum.json"

บวก(2,3)

โมดูลสำเร็จรูป

โมดูลสำเร็จรูปคือ โมดูลที่ติดมาพร้อมกับภาษา THL โดยการนำเข้าโมดูลประเภทนี้จะทำแบบเดียวกับกับการนำเข้าแบบปกติโดยไม่ต้องสร้างโมดูลขึ้นมาก่อน

ตัวอย่าง:

นำเข้า "math.json"

แสดง(abs(-67),"\\n")

แสดง(gcd(12,3))

ตัวอย่างเพิ่มเติม

ตัวอย่างการสร้างไฟล์ส่งออก

ตัวอย่าง:

นำเข้า "type.json"

ส่งออก โปรแกรม จำนวนเฉพาะ(n):

prime คือ []

ans คือ []

สำหรับ i ในช่วง(n+1):

prime.เพิ่ม(จริง)

สำหรับ p ในช่วง(2,เปลี่ยนจำนวนเต็ม(2 ราก n) + 1):

ถ้า prime[p]:

สำหรับ i ในช่วง(p*p , n+1 , p):

prime[i] คือ เท็จ

สำหรับ i ในช่วง(2,n):

ถ้า prime[i] = จริง:

ans.เพิ่ม(i)

คืนค่า ans

ตัวอย่างการนำเข้าไฟล์

ตัวอย่าง:

นำเข้า "sieve.json"

แสดง(จำนวนเฉพาะ(100))

ภาคผนวก ค ข้อตกลงในการใช้ซอฟต์แวร์

ข้อตกลงในการใช้ซอฟต์แวร์

ซอฟต์แวร์นี้เป็นผลงานที่พัฒนาขึ้นโดย นายภวัต มติธรรม จาก โรงเรียนเบญจมราชูทิศ ภายใต้การ ดูแลของ นางสาวธิยาภรณ์ ศรีเมือง ภายใต้โครงการ ภาษาคอมพิวเตอร์ที่เป็นภาษาไทย ซึ่งสนับสนุนโดย สำนักงาน พัฒนา วิทยาศาสตร์และเทคโนโลยีแห่งชาติ โดยมีวัตถุประสงค์เพื่อส่งเสริมให้นักเรียนและ นักศึกษาได้เรียนรู้และฝึก ทักษะในการพัฒนาซอฟต์แวร์ ลิขสิทธิ์ของซอฟต์แวร์นี้จึงเป็นของ ผู้พัฒนา ซึ่งผู้พัฒนาได้อนุญาตให้สำนักงาน พัฒนาวิทยาศาสตร์และเทคโนโลยีแห่งชาติ เผยแพร่ซอฟต์แวร์นี้ตาม "ต้นฉบับ" โดยไม่มีการแก้ไขดัดแปลงใด ๆ ทั้งสิ้น ให้แก่บุคคลทั่วไป ได้ใช้เพื่อประโยชน์ส่วนบุคคลหรือประโยชน์ทางการศึกษาที่ไม่มีวัตถุประสงค์ในเชิง พาณิชย์ โดยไม่คิดค่าตอบแทนการใช้ซอฟต์แวร์ ดังนั้น สำนักงานพัฒนาวิทยาศาสตร์และเทคโนโลยี แห่งชาติ จึงไม่ มีหน้าที่ในการดูแล บำรุงรักษา จัดการอบรมการใช้งาน หรือพัฒนา ประสิทธิภาพซอฟต์แวร์ รวมทั้งไม่รับรองความ ถูกต้องหรือประสิทธิภาพการทำงานของ ซอฟต์แวร์ ตลอดจนไม่รับประกันความเสียหายต่าง ๆ อันเกิดจากการใช้ ซอฟต์แวร์นี้ทั้งสิ้น

License Agreement

This software is a work developed by Mr.Phawat Matitham from Benjamarachutit School under the provision of Miss Thiyaporn Srimuang under Thai-Based Programming Language, which has been supported by the National Science and Technology Development Agency (NSTDA), in order to encourage pupils and students to learn and practice their skills in developing software. Therefore, the intellectual property of this software shall belong to the developer and the developer gives NSTDA a permission to distribute this software as an "as is" and non-modified software for a temporary and non-exclusive use without remuneration to anyone for his or her own purpose or academic purpose, which are not commercial purposes. In this connection, NSTDA shall not be responsible to the user for taking care, maintaining, training, or developing the efficiency of this software. Moreover, NSTDA shall not be liable for any error, software efficiency and damages in connection with or arising out of the use of the software."